

APPLIED MATH FOR COMPUTER SCIENCE

CPCS 212

Lab 9: Numerical Integration and Differentiation

Open the diary file
Lab9_yourname.txt

Agenda

- Symbolic Math Toolboxes
 - Symbolic Integration
 - Symbolic Differentiation
- Numerical Integration
- Numerical Differentiation

Symbolic Math Toolboxes

- Symbolic Math Toolbox provides functions for solving and manipulating symbolic math expressions and performing variable-precision arithmetic. You can analytically perform differentiation, integration, simplification, transforms, and equation solving.

Symbolic Math Toolboxes

Symbolic Math Toolboxes incorporate symbolic computation into the numeric environment of MATLAB.

Variables must be define as symbolic by **sym** or **syms**

```
>> x = sym('x'), y = sym('y'), z = sym ('z')
```

or

```
>> syms x y z
```

Symbolic Integration with MATLAB

Integration Built-in Function in Old Version of Matlab

```
>> I = int(f)      %Indefinite integral  
>> I = int(f, v)  %Designating integration variable  
>> I = int(f, a, b) %Definite integral on close interval  
>> I = int(f, v, a, b)
```

Variables must be define as symbolic by sym or syms
where f = symbolic expression

Symbolic Integration with MATLAB

Integration Built-in Function in New Version of Matlab

```
>> I = integral(f,a,b)
```

f= the function

a,b=the interval boundaries

Must use function handle @ to define the function and the variables.

Example:

Evaluate the integral $\int_1^2 x \cos x dx$

```
>>f=@(x)x.*cos(x)
```

```
>>l=integral(f,1,2)
```

```
>>l =
```

```
0.0207
```


Example:

Evaluate the integral
 $f=x^2-9$ in the interval $[0,5]$

```
>>f=@(x)x.^2-9
```

```
>>l=integral(f,0,5)
```

```
>> l =
```

```
-3.3333
```

Numerical Integration

Trapezoidal numerical integration

MATLAB's trapz function

>> z = trapz(y) % Integral of y with unit spacing
>> z = trapz(x, y) % Integral of y with respect to x

```
>> y = [1 4 9 16 25] ;
```

```
>> z = trapz(y)
```

```
z =
```

```
42
```

```
>> x = 0 : pi/100 : pi;
```

```
>> y = sin(x);
```

```
>> z = trapz(x,y)
```

```
z =
```

```
1.9998
```

Example 1:

Use MATLAB's **trapz** function to numerically integrate

$Y = \sin(x) - x^2$;

from $a = 0$ to $b = 2$.

```
>>X =linspace(0,2*pi,1000);
```

```
>>Y = sin(X) - X.^2;
```

```
>>Z = trapz(X,Y)
```

```
Z = -82.6834
```

Example 2:

Use MATLAB's **trapz** function to numerically integrate

$$f(x) = 0.2 + 25x - 200x^2 + 675x^3 - 900x^4 + 400x^5$$

from $a = 0$ to $b = 0.8$. Note that the exact value of the integral can be determined analytically to be 1.640533.

Solution

```
>> x = linspace(0, 0.8, 3);  
>> fx = 0.2+25*x-200*x.^2+675*x.^3-  
        900*x.^4+400*x.^5;  
>> l = trapz(x,fx)  
l =  
    1.0688
```

Trapezoidal numerical integration method

Algorithm

Input: Continuous function $f(x)$

Interval ends a, b

Output: Definite integral of function $f(x)$ in the interval $[a, b]$

Steps:

1. Divide the interval into subintervals with a step h

$$P = \{a = x_0 \leq x_1 \leq x_2 \leq \dots \quad b\}$$

2. $S = 0$

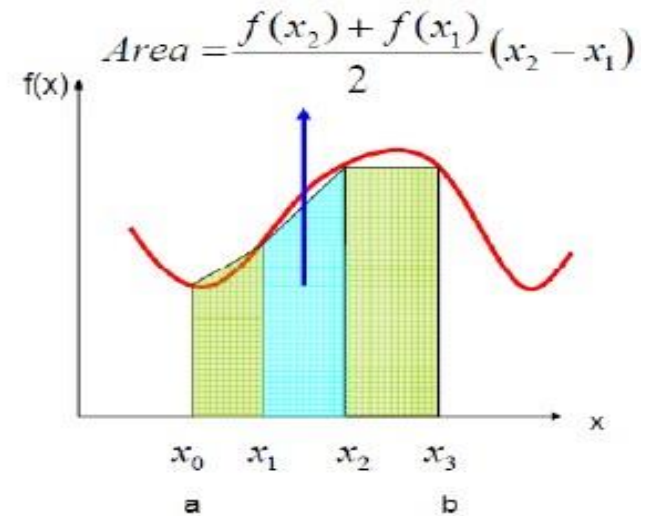
3. Loop $i = 0 \rightarrow n-1$

1. Compute $f(x_i)$ and $f(x_{i+1})$

2. Add to S $f(x_i) + f(x_{i+1})$

End loop

4. Calculate the integral $S = h \frac{S}{2}$



$$h = \frac{b - a}{n}$$

$$x_i = a + ih$$

Trapezoid2.m

```
function s = trapezoid2(f,a,b,n)
s=0;
h=(b-a)/n;
%iterate from the beginning of the interval (a) to the
beginning of the last interval (b-h)
for i=a:h:b-h
    x=i;    f1=feval(f,x);
    x=i+h;  f2=feval(f,x);
    s=s+f1+f2;
end
s=h*s/2;
end
```


Exercise 1

Use **Trapezoid2** to numerically integrate $y = x^2$
from $a=0$ to $b=1$ and $n=4$

```
function y=f(x)
    y = x^2;
end
```

```
>> s = trapezoid2('y',0,1,4)
s =
    0.3438
```

OR you can use function handler

```
>>y=@(x)x^2;
```

```
>> s = trapezoid2(y ,0,1,4)
s =
    0.3438
```

Numerical Differentiation

Symbolic Differentiation with MATLAB

% differentiates a symbolic expression S with respect to its free variable as determined by symvar.

>> diff(S)

%differentiates S with respect to v.

>> diff(S,'v') or diff(S,sym('v'))

%for a positive integer n, differentiates S, n times.

>> diff(S,n)

%differentiates S with respect to v , n times.

>>diff(S,'v',n) and diff(S,n,'v')

%Variables must be define as symbolic by sym or syms

Examples

```
>> syms x t
>> diff(sin(5*x))    % ans= 5*cos(5*x) %the first derivative
>> diff(t^3)         % ans= 3*t^2  %the first derivative
>> diff(t^3,2)       % ans= 6*t  %the second derivative
>> diff(t^3,3)       % ans= 6  %the third derivative
>> diff(t^3,4)       % ans= 0  %the fourth derivative
>> diff(sin(x*t),t)  % ans= cos(x*t)*t
```

Numerical Differentiation with MATLAB

%calculates differences between adjacent elements of X.

```
>>Y = diff(X)
```

%applies diff recursively n times, resulting in the nth difference. Thus, **diff(X,2)** is the same as **diff(diff(X))**.

```
>>Y = diff(X,n)
```

%the n th difference function calculated along the dimension specified by scalar dim.

```
>>Y = diff(X,n,dim)
```

Numerical Differentiation with MATLAB

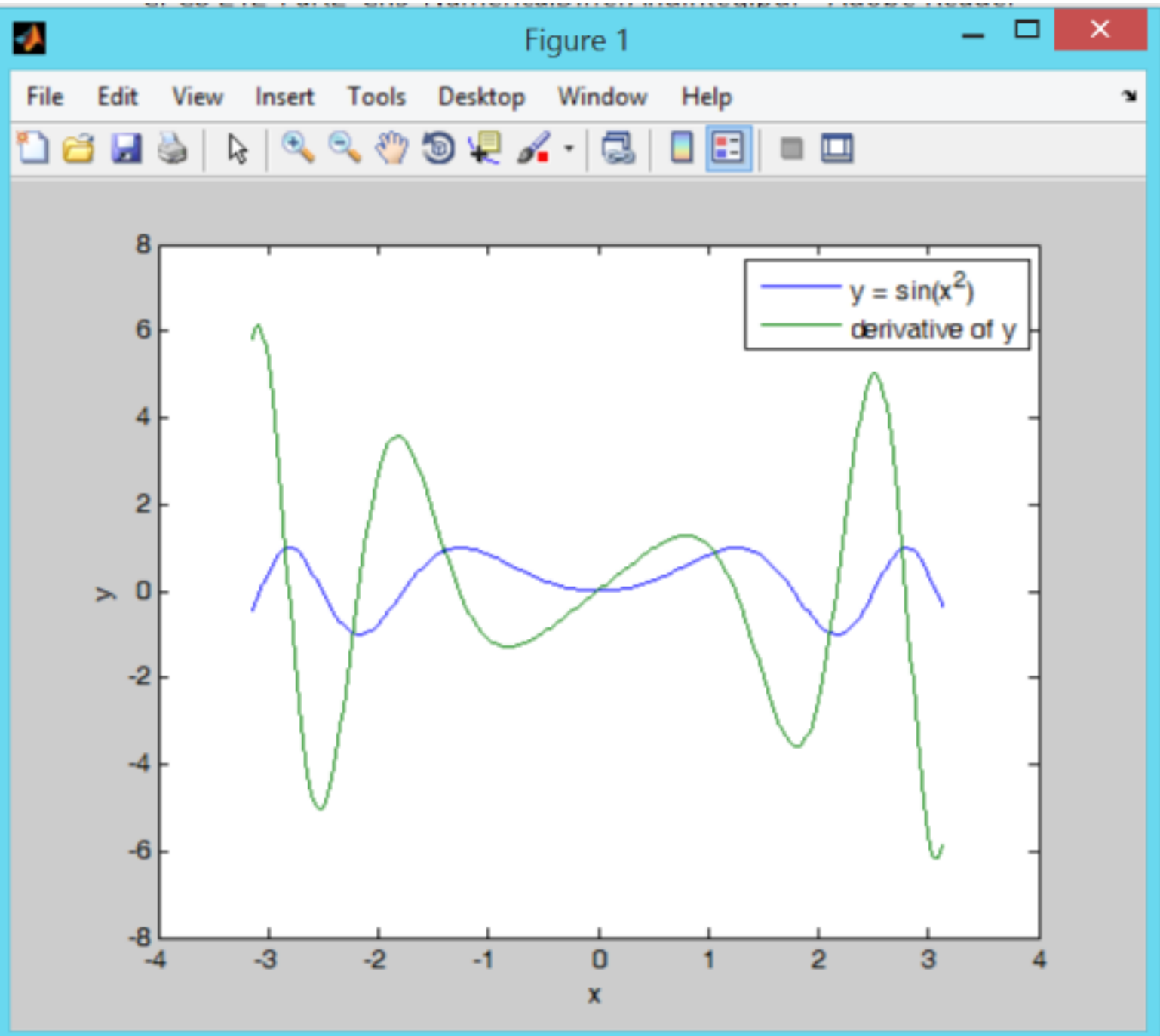
Note:

The quantity **`diff(y)./diff(x)`** is an approximate derivative.

- When using **`yp=diff(y)./diff(x)`** you need to map it to an associated **`x`** value. Since you are, in essence, calculating the slope of a line segment, you can choose to map it to the beginning point (forward difference), ending point (backward difference), or the midpoint (central difference).
- To generate the associated **`xp`** values for **`yp`**:
 - use **`xp=x(1:end-1)`** for forward difference
 - use **`xp=x(2:end)`** for backward difference
 - use **`xp=x(1:end-1) + diff(x)./2`** for central difference

Numerical Differentiation Example

```
>> deltax = 0.02;  
>> x = -pi:deltax:pi;  
>> y = sin(x.^2);  
>> yprime = diff(y)./diff(x);  
  
>> plot(x,y, x(1:end-1),yprime)  
>> legend('y = sin(x^2)','derivative of y')  
>> xlabel('x'),ylabel('y')
```



Numerical Differentiation

1. First order derivatives (difference method)

Input: Continuous function $f(x)$, Derivative point x , Derivative step h

Output: First derivative of function $f(x)$ at x

Steps:

1. Compute $f(x)$, $f(x+h)$ and $f(x-h)$

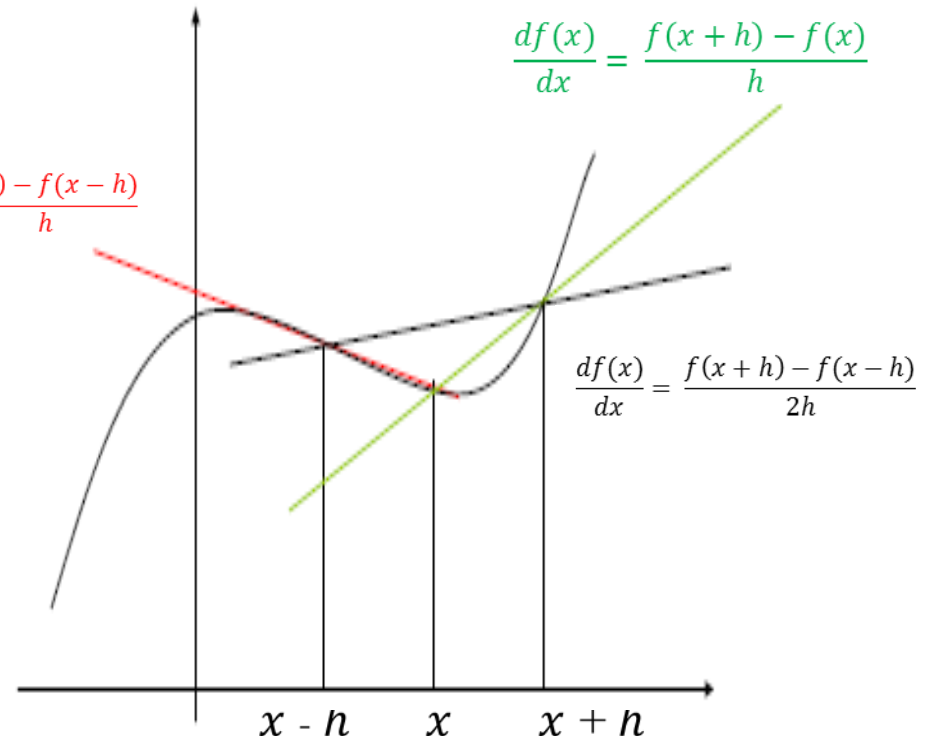
2. If Forward method $df = \frac{f(x+h) - f(x)}{h}$

If Backward method $df = \frac{f(x) - f(x-h)}{h}$

$$\frac{df(x)}{dx} = \frac{f(x) - f(x-h)}{h}$$

$$\frac{df(x)}{dx} = \frac{f(x+h) - f(x)}{h}$$

$$\frac{df(x)}{dx} = \frac{f(x+h) - f(x-h)}{2h}$$



df1.m

```
function d = df1(f,xi,h,method)

% First Numerical Differentiation using Difference Method

x=xi ;    f1=feval(f,x);
x=xi+h ;  f2=feval(f,x);
x=xi-h ;  f0=feval(f,x);

if strcmp(method,'f')==1      % method == 'f' forward difference
    d=(f2-f1)/h;
elseif strcmp(method,'b')==1 % method == 'b' backward difference
    d=(f1-f0)/h;
elseif strcmp(method,'c')==1 % method == 'c' central difference
    d=(f2-f0)/2/h;
else
    d='unknown method';
end
```

Call your function

```
>> y= @(x)x^2-2*x;  
>>d=df1(f,1,0.001,'f')
```

```
d = 1.0000e-003
```

```
>> y= @(x)x^2-2*x;  
>> d=df1(y,1,0.001,'r')
```

```
d = unknown method
```

```
>> y= @(x)x^2-2*x;  
>> d=df1(y,1,0.001,'b')
```

```
d = -0.0010
```

```
>> y= @(x)x^2-2*x;  
>> d=df1(y,1,0.001,'c')
```

```
d = -5.5511e-014
```

Numerical Differentiation

2. Second order derivatives(difference method)

Algorithm

Input: Continuous function $f(x)$, Derivative point x , Derivative step h

Output: Second derivative of function $f(x)$ at x

Steps:

1. Compute $f(x)$, $f(x+h)$ and $f(x-h)$

2. Compute $d^2f = \frac{f(x+h) - 2f(x) + f(x-h)}{h^2}$

df2.m

```
function d=df2(f,xi,h)
```

```
% Second Numerical Differentiation using  
Difference Method
```

```
x = xi ; f1=feval(f,x);
```

```
x=xi+h; f2=feval(f,x);
```

```
x=xi-h; f0=feval(f,x);
```

```
d=(f2-2*f1+f0)/(h*h);
```

```
>>y= @(x)x^2-2*x;
```

```
>> d=df2(y1,0.001)
```

```
d =  
    2.0000
```